

Desarrollo de software embebido a partir de contenedores

Caso Docker

Verónica Cambronero Solano

Instituto Tecnológico de Costa Rica
Taller de Sistemas Embebidos

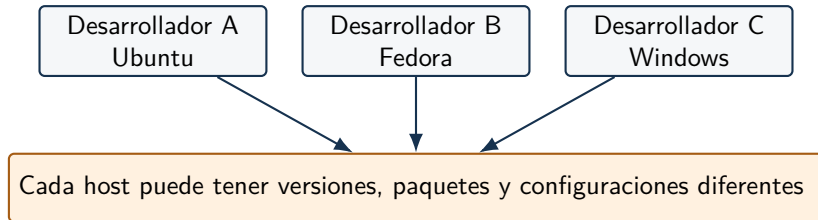
2026

Diagnóstico inicial

- ❶ **V/F:** Un contenedor Docker es esencialmente una máquina virtual pequeña.
- ❷ **V/F:** Un contenedor Linux normalmente utiliza el kernel del host.
- ❸ **V/F:** Una imagen Docker y un contenedor son exactamente lo mismo.
- ❹ Una laptop amd64 debe ejecutar una aplicación en un target arm64. ¿Podemos asumir que el mismo binario funcionará en ambos?
- ❺ ¿Docker tiene que ejecutarse necesariamente en el dispositivo embebido para ser útil durante su desarrollo?

No buscamos acertar todo: estas preguntas definen el problema que vamos a resolver.

El problema: “funciona en mi máquina”



Mismo proyecto

Código fuente y repositorio compartidos.

Entorno no necesariamente igual

Compilador, SDK, librerías, dependencias y versiones pueden variar.

Mismo código no implica necesariamente mismo entorno de construcción.

Fuente: Docker Docs; Yocto Project – Development Environment

¿Un contenedor es una VM pequeña?

Máquina virtual

Aplicación + librerías
Sistema operativo invitado
Kernel propio
Hipervisor
Hardware

Cada VM dispone de su propio kernel.

Contenedor Linux

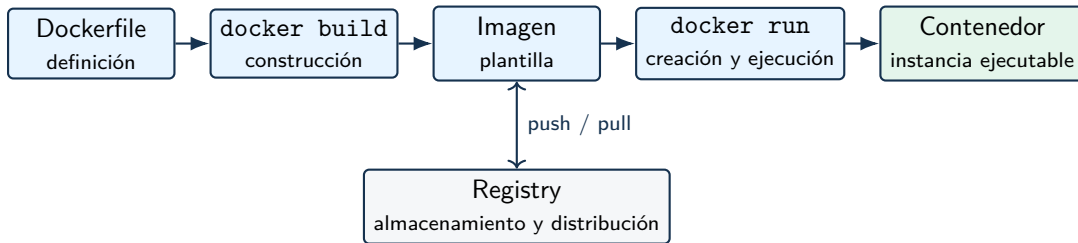
Aplicación + librerías
Proceso aislado
Kernel compartido del host
Hardware

Los procesos utilizan el kernel del host.

Una VM virtualiza una máquina completa; un contenedor aísla procesos y su entorno de usuario utilizando el kernel del host.

Fuente: Docker Docs – What is a container?

¿Qué aporta Docker?



Imagen

Plantilla de solo lectura construida en capas.

Contenedor

Instancia ejecutable creada a partir de una imagen.

Docker conecta la definición del entorno con su construcción, distribución y ejecución.

Fuente: Docker Docs – Docker Overview; Dockerfile Reference

Si comparten el kernel, ¿cómo se aíslan?

Namespaces

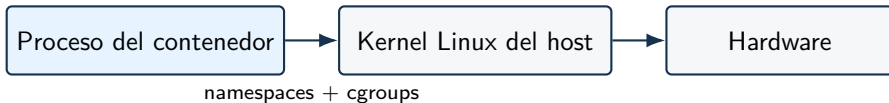
¿Qué puede ver?

- ▶ pid: procesos
- ▶ network: red
- ▶ mount: puntos de montaje

cgroups

¿Cuánto puede usar?

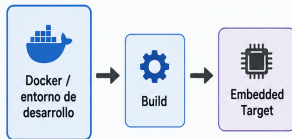
- ▶ CPU
- ▶ RAM
- ▶ PIDs / tareas



Fuente: OCI Runtime Specification – Linux configuration

Dos usos distintos de contenedores en Embedded

A. Docker en el Development Host

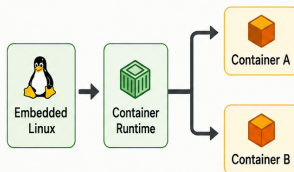


Docker se utiliza para construir el software



El target no necesita ejecutar Docker

B. Contenedores también en el Target



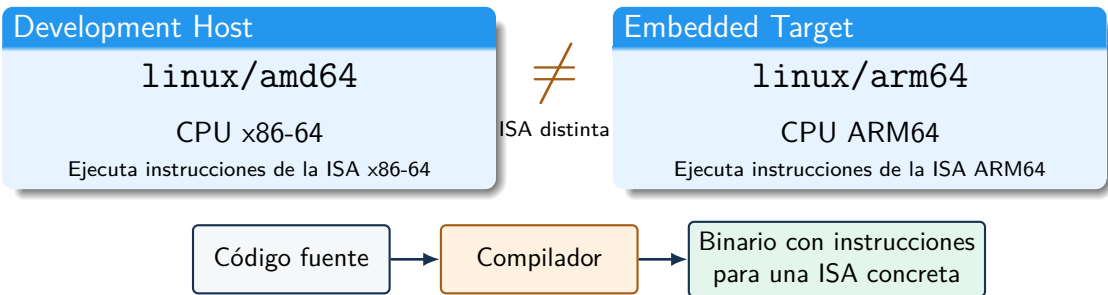
Los contenedores forman parte del sistema final



Aquí sí importan *runtime*, aislamiento y overhead

Usar Docker para desarrollar software embebido no es lo mismo que ejecutar contenedores dentro del target.

Mismo Linux $\neq$ misma arquitectura



BUILD PLATFORM y TARGET PLATFORM pueden ser diferentes; el binario debe ser compatible con la ISA del target.

Fuente: Docker Docs – Multi-platform Builds

Una referencia, varias plataformas



¿Qué ocurre al hacer `docker pull`?

Docker identifica la plataforma del host y selecciona la variante compatible.

Una imagen multi-platform mantiene una sola referencia que apunta a variantes específicas para cada plataforma.

Fuente: Docker Docs – Multi-platform Builds

Tres estrategias para construir para otra arquitectura

Estrategia	Cómo funciona	Ventaja	Trade-off
QEMU	Emula la arquitectura objetivo	Fácil de comenzar	Puede ser mucho más lento en compilación
Nodos nativos	Cada arquitectura construye sobre hardware propio	Rendimiento nativo	Más infraestructura y administración
Cross-compilation	El compilador corre en el host y genera código para otra ISA	Evita emular la compilación	Requiere soporte del lenguaje/toolchain

Fuente: Docker Docs – Multi-platform Builds

BuildKit + Multi-stage: de AMD64 a ARM64

```
FROM --platform=$BUILDPLATFORM golang:alpine AS
    build

ARG TARGETOS
ARG TARGETARCH

WORKDIR /src
COPY . .

RUN GOOS=$TARGETOS GOARCH=$TARGETARCH \
    go build -o app .

FROM alpine
COPY --from=build /src/app /app
```

Etapa 1 — Builder

`BUILDPLATFORM = linux/amd64`

Compilador Go

`GOARCH=arm64`



Binario ARM64

Etapa 2 — Imagen final

`TARGETPLATFORM = linux/arm64`

`COPY --from=build`



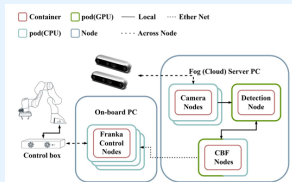
Aplicación ARM64

Clave: TARGETARCH indica la arquitectura objetivo, pero el compilador debe soportar cross-compilation.

Fuente: Docker Docs – Dockerfile Reference: Multi-platform Builds

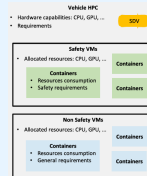
Contenedores en el target: dos escalas muy distintas

Cloud-Native Fog Robotics



- ▶ Franka Panda + On-board PC + Fog Server
- ▶ ROS + K3s
- ▶ Microservicios contenerizados

Software-Defined Vehicle



- ▶ ADLINK AVA AP1
- ▶ ARM, 32 cores, 32 GB RAM
- ▶ VMs + K3s + contenedores

“Embedded” no implica una única escala de hardware: los recursos disponibles condicionan la arquitectura.

Beneficios y overhead: evidencia experimental

Beneficios/capacidades reportadas

- ▶ Modularización de componentes
- ▶ Aislamiento entre servicios
- ▶ Gestión de recursos
- ▶ Flexibilidad para actualizar componentes

37 %

más lento en una tarea simple de CPU
evaluada

3 %

más lento en una tarea simple de GPU
evaluada

2.9 s

para asignar el deployment y crear
contenedores en Raspberry Pi

Resultados de experimentos específicos; no son valores universales de Docker.

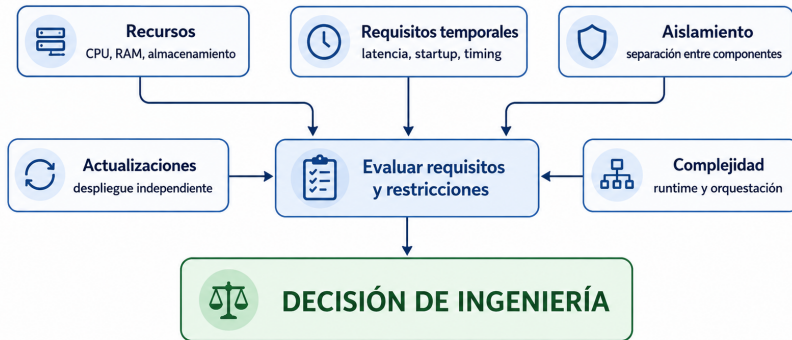
Fuente: Cloud-Native Fog Robotics

Aspectos evaluados

- ▶ Uso de CPU y RAM
- ▶ Tiempo de inicio
- ▶ Tiempo de ejecución
- ▶ Latencia de comunicación



¿Tiene sentido ejecutar contenedores en este target?



La decisión de ejecutar contenedores en el target depende de los requisitos y restricciones del sistema.

Fuente: Síntesis propia a partir de Docker, OCI, Yocto Project y casos de estudio

Respuestas del diagnóstico

- ❶ **Falso.** Un contenedor no es simplemente una máquina virtual pequeña; en Linux comparte el kernel del host.
- ❷ **Verdadero.** Un proceso ejecutado dentro de un contenedor Linux utiliza el kernel del sistema host.
- ❸ **Falso.** La imagen es una plantilla de solo lectura; el contenedor es una instancia ejecutable creada a partir de ella.
- ❹ **No.** Un binario compilado para amd64 no es nativamente compatible con una CPU arm64.
- ❺ **No necesariamente.** Docker puede utilizarse únicamente en el *Development Host* para crear el entorno de desarrollo y construir software para el *Embedded Target*.

¿Qué dudas o comentarios tienen hasta este punto?

Antes de pasar a la demostración

Referencias principales I

 Docker Documentation. *Docker overview / What is Docker?*.
docs.docker.com/get-started/docker-overview/

 Docker Documentation. *What is a container?*.
Docker Docs – What is a container?

 Docker Documentation. *Dockerfile reference*.
docs.docker.com/reference/dockerfile/

 Docker Documentation. *Multi-platform builds*.
docs.docker.com/build/building/multi-platform/

 Yocto Project Documentation. *Development Environment*.
docs.yoctoproject.org/.../development-environment.html

 Open Container Initiative. *Runtime Specification*.
github.com/opencontainers/runtime-spec

 L. Wen, Y. Zhang, M. Rickert, J. Lin, F. Pan and A. Knoll, “Cloud-Native Fog Robotics: Model-Based Deployment and Evaluation of Real-Time Applications,” *IEEE Robotics and Automation Letters*, vol. 10, no. 1, pp. 398–405, Jan. 2025, doi: 10.1109/LRA.2024.3504243.

Referencias principales II



F. Pan et al., "Toward Software-Defined Vehicles: From Model-Based Engineering to Virtualization-Based Deployment," *IEEE Access*, vol. 12, pp. 192127–192145, 2024, doi: [10.1109/ACCESS.2024.3512002](https://doi.org/10.1109/ACCESS.2024.3512002).